CH-8 PART - 1

In-Depth Study Notes: Oracle SQL Set Operators and Compound Statements

As an expert Oracle SQL Developer, I have synthesized the execution logs provided in your latest files (CH8-1.TXT and CH8-2.TXT). When business requirements demand comparing, combining, or finding the differences between entirely separate datasets, standard joins are not always applicable. These notes detail the architecture of **Set Operators**, **Compound Precedence**, and **Matrix Logic Translation**.

Concept 1: Vertical Data Combination (Set Operators)

Core Purpose: Unlike JOINs, which combine tables horizontally (adding columns side-by-side), **Set Operators** combine result sets *vertically* (stacking rows on top of each other) 1. When two or more SELECT statements are joined using a set operator, the entire script is known as a **Compound Statement** 1.

Keyword Breakdown:

- **UNION ALL**: Simply clubs both result sets vertically, maintaining the exact top-to-down random order of rows, and **keeps all duplicate rows** exactly as they are 1.
- **UNION**: Clubs the result sets together, but performs two implicit overhead tasks: **1) Removes duplicate rows, and 2) Sorts the final result set in ascending order** 2.
- **INTERSECT**: Extracts only the rows that are **common** to both result sets, then removes duplicates and sorts the output in ascending order 3.
- **MINUS**: Acts as a subtractor. It completely removes any common rows from the **first** result set, and outputs only the remaining unique rows from the top query, sorted in ascending order 4.

Comparison of Set Operators:

Operator,Deduplicates Data?,Sorts Output?,Execution Speed,Business Use Case
UNION ALL,No,No,Fastest,Combining massive archives where duplicates are allowed.
UNION,Yes,Yes,Slower (CPU Sort),Generating a distinct master list from multiple tables.
INTERSECT,Yes,Yes,Slower,Finding employees who exist in both the 1998 and 1999 hiring cohorts 5.
MINUS,Yes,Yes,Slower,Finding employees in Dept 10 who do not share a Job ID with Dept 20 4.

Syntax Breakdown:

```
SELECT column_1, column_2 FROM table_a
[ SET OPERATOR ]
SELECT column_1, column_2 FROM table_b;
```

Detailed Example: The HR manager needs a report showing all unique DEPARTMENT_IDs for which the company pays salaries of $6,000 or more, explicitly excluding any departments that *also* pay salaries strictly under $6,000.

```
-- Query 1 retrieves all departments paying high salaries
```

```
SELECT DEPARTMENT_ID FROM EMPLOYEES WHERE SALARY >= 6000
MINUS
-- Query 2 subtracts departments paying low salaries
SELECT DEPARTMENT_ID FROM EMPLOYEES WHERE SALARY < 6000;
```

[Execution: Outputs only the distinct departments that exclusively pay 6000 or more 6.]

Execution Logic:

1. **Independent Evaluation**: The Oracle Engine executes the top query and the bottom query completely independently, caching their result sets in the PGA (Program Global Area).
2. **Validation**: The engine verifies that both queries have the **exact same number of columns** and that the corresponding columns have **compatible data types**.
3. **Set Operations**: If using UNION/INTERSECT/MINUS, the CPU performs an aggressive hash or sort operation to identify duplicates and order the final output before projecting it to the user.

Pro-Tips/Best Practices:

- **Always default to UNION ALL**: Sorting is one of the most expensive operations for a database CPU. If you know your data sets do not contain duplicates (e.g., combining JAN_SALES and FEB_SALES), use UNION ALL. Using UNION will force an unnecessary Full Table Sort, destroying query performance.
- **Avoiding SELECT ***: Never use SELECT * in compound statements. If a DBA adds a column to table_a but not table_b, the column counts will mismatch, and the script will instantly crash with an **ORA-01789: query block has incorrect number of result columns**. Always list columns explicitly.

Concept 2: Compound Statement Precedence

Core Purpose: When chaining three or more Set Operators together, the execution order dictates the final output. Without explicit guidance, Oracle evaluates Set Operators strictly from **top-left to down-right** 7.

Keyword Breakdown:

- **() (Parentheses)**: Used to override the default top-to-bottom execution precedence, forcing the engine to evaluate specific SELECT blocks first 7.

Comparison of Precedence Behaviors:

Query Structure,First Action,Second Action,Final Row Count (Example Data)
10 INTERSECT 20 UNION 30,Intersects 10 & 20 first.,Unions the result with 30.,3 Rows 8
10 INTERSECT (20 UNION 30),Unions 20 & 30 first.,Intersects 10 with that combined result.,2 Rows 8

Syntax Breakdown:

```
SELECT ... FROM ...
SET OPERATOR
(
    SELECT ... FROM ...
    SET OPERATOR
    SELECT ... FROM ...
);
```

Detailed Example: We want to combine Department 20 and 30 into a single master pool first, and then find which Job IDs in Department 10 do *not* exist in that combined pool.

```
SELECT JOB FROM MPSEMP WHERE DEPTNO = 10
MINUS
(
    SELECT JOB FROM MPSEMP WHERE DEPTNO = 20
    UNION 30
    SELECT JOB FROM MPSEMP WHERE DEPTNO = 30
);
```

[Execution: The engine first builds the deduplicated list of jobs from Dept 20 and 30, then subtracts those jobs from the Dept 10 list 9.]

Pro-Tips/Best Practices:

- **Always use parentheses for chained sets:** Just like mathematical operations (AND/OR), writing chained Set Operators without parentheses makes the business logic incredibly difficult for other developers to read and debug.

Concept 3: Generating Matrix Reports (Set Operators vs. Subqueries)

Concept: Generating Matrix Reports via Compound Statements

Core Purpose: This method builds a complete Cartesian product of all possible combinations, assigns a default negative flag (`'NO'`) to everything, physically subtracts the combinations that are actually true using `MINUS`, and then re-inserts those true combinations back into the result set with a positive flag (`'YES'`) using `UNION`.

Keyword Breakdown:

- **`CROSS JOIN`**: Multiplies every row in Table A by every row in Table B to create the foundational grid.
- **`MINUS`**: The subtractor. It identifies the staff/subject pairs that do exist in the capabilities table and rips them out of the baseline 'NO' grid.
- **`UNION` / `UNION ALL`**: Appends the valid capabilities back into the final result set, but with the updated 'YES' string.
- **Literal Strings (`'YES '` / `'NO '`)**: Hardcoded text injected directly into the `SELECT` projection to act as dynamic flags.

Comparison of Matrix Mapping Logic:

Action Phase	SQL Operator	Row Count Reality (Example Data)	Business Meaning
1. Baseline Generation	`CROSS JOIN`	12 Rows (All `'NO '`)	Assumes no one can teach anything.

2. Subtraction	`MINUS`	8 Rows (True `'NO '`s)	Removes the 4 combinations that the staff are actually certified to teach.
3. Re-insertion	`UNION`	12 Rows (8 `'NO'`, 4 `'YES'`)	Injects the 4 certified combinations back into the list with a `'YES'` flag.

Syntax Breakdown:

```
SELECT col_A, col_B, 'DEFAULT_FLAG' AS Alias
FROM table_A CROSS JOIN table_B
MINUS
SELECT col_A, col_B, 'DEFAULT_FLAG'
FROM mapping_table
UNION
SELECT col_A, col_B, 'NEW_FLAG'
FROM mapping_table;
```

Detailed Example: An institute needs a master grid of all staff and all subjects (`MPSSTAFF` x `MPSSUBJECT`), verifying their abilities against the `MPSCANTEACH` table using strictly set operators.

```
-- Step 1: Create the baseline matrix with all 'NO's
SELECT T.NAME AS STAFF , B.NAME AS SUBJECT , 'NO  ' AS "CAN_TEACH?"
FROM MPSSTAFF T CROSS JOIN MPSSUBJECT B
-- Step 2: Subtract the valid pairs from the 'NO' list
MINUS
SELECT STAFF , SUBJECT , 'NO  '
FROM MPSCANTEACH
-- Step 3: Add the valid pairs back with 'YES'
UNION
SELECT STAFF , SUBJECT , 'YES '
FROM MPSCANTEACH;
```

[Execution: The engine generates 12 rows. Ashwin-TOEFL is initially tagged 'NO', stripped out by the MINUS, and added back as 'YES'. BIMAL-GRE is tagged 'NO', unaffected by the MINUS, and remains 'NO' 20, 21, 22, 26].

Execution Logic: By the rule of compound precedence, the Oracle Engine evaluates this strictly from **top-left to down-right**.

1. It executes the `CROSS JOIN` to build the 12-row baseline matrix.

2. It executes the first subquery (`MPSCANTEACH` with 'NO') and performs the `MINUS`, reducing the cached result set to 8 rows.
3. It executes the final subquery (`MPSCANTEACH` with 'YES') and `UNION`s it to the cached result, outputting the final 12 categorized rows.

Pro-Tips/Best Practices:

- **The String Padding Rule:** Notice the syntax strictly uses `'NO '` (with two trailing spaces) and `'YES '` (with one trailing space). Because set operator queries must have matching data types and lengths (Rule 2), you must pad your literal strings with spaces so both outputs are exactly 4 characters long. If you use `'NO'` and `'YES'`, the query may fail or truncate data due to length mismatch across the compound blocks.
- **Performance Hit:** While this architecture works flawlessly, **it is not best practice**. Using `MINUS` and `UNION` forces the database engine to scan the tables multiple times and perform heavy CPU sorting. The `CASE WHEN ... IN (Subquery)` method (your original query) is highly preferred because it achieves the exact same output with only a single table scan.

RULES FOR USING SET OPERATORS :

- ALL QUERIES SHOULD HAVE SAME NUMBER OF ITEMS.
- THEIR CORRESPONDING DATA TYPES SHOULD MATCH.
- COLUMN ALIAS USED IN FIRST QUERY WILL BE DISPLAYED IN RESULT.
- IF WE WANT TO USE OFFSET OR FETCH FOR INDIVIDUAL STATEMENT, EACH STATEMENT MUST BE PUT WITHIN ()
- IF WE WANT TO USE ORDER BY CLAUSE FOR INDIVIDUAL STATEMENT, THEN EITHER OFFSET OR FETCH IS MANDATORY. EACH STATEMENT MUST BE PUT WITHIN ().
- WHEN WE USE */META CHARACTER IN SELECT AND IF WE USE ORDER BY AT COMPOUND STATEMENT LEVEL, ORACLE ALLOWS COLUMN REFERENCE ONLY WITH IT'S POSITION.